Software Project II

TA. Menna Singergy

Spring 2026

1 Project Overview

1.1 Description:

The Real-Time Study Buddy Matcher is a full-stack, microservices-based system designed to help students find compatible study partners or groups based on shared academic interests, availability, and study preferences. Many students struggle to study effectively alone or to find peers who match their pace and schedule. This platform solves that problem by dynamically matching students using real-time data and event-driven communication through sessions created that can be either in-person (offline) or online. Offline sessions can be reserved in university study rooms.

The main features in the project are:

- User and Authentication Service
- Profile and Study Preferences Service
- Availability Management Service
- Matching Service
- Study Session Scheduling Service
- Notification Service
- Messaging / Chat Service (Bonus Feature)

1.2 Team Formation:

The team should strictly consist of 5 members.

1.3 Technology, Architecture and Stack:

The system is built using a **microservices architecture**, where each service is independently developed and responsible for a specific business capability. Communication between services is handled asynchronously using Kafka, while a GraphQL Gateway provides a unified API for the frontend. Each service manages its own data using NeonDB (serverless PostgreSQL), and kafka. Finally, the project will be deployed on Docker and on Render/Railway.

The system will be built using **React (Frontend)** and **Node.js (Backend)** with **GraphQL API (Apollo), NeonDB (Serverless PostgreSQL)** for database management, and **Prisma ORM (Object-relational mapping)** for seamless interactions.

1.4 Structure:

The project is divided into **three milestones, covering UI design, backend development, and frontend integration and full project deployment.**

2 Functional & Non-Functional Requirements

These are the main FR and NFR that **MUST** be fulfilled by the system.
Note: you can add additional FR and NFR if needed, but these are the essential ones.

2.1 User and Authentication Service

2.1.1 Functional Requirements:

- The system must allow users to register a new account using basic information such as name, email, and password.
- The system must allow users to log in using their email and password.
- The system must generate an authentication token (for example JWT) after successful login.
- The system must allow authenticated users to retrieve their profile information.
- The system must allow users to update their basic profile details.

2.1.2 Non-functional Requirements:

- Passwords must be securely stored using hashing (e.g., bcrypt).
- Authentication tokens must be used to protect graphQL API.
- The service must ensure data privacy and prevent unauthorized access to user accounts.

2.2 Profile and Study Preferences Service

2.2.1 Functional Requirements:

- The system must allow users to add and update courses they are studying.
- The system must allow users to define study preferences, such as:
  1. preferred study pace
  2. preferred study mode (online or in-person)
  3. preferred group size
  4. preferred study style (e.g., writing notes, listening, discussing out loud, studying quietly, or other methods)
- The system must allow users to add study topics or subjects they need help with.
- The system must allow users to view and update their study preferences at any time.
- The service must publish an event to Kafka whenever preferences or courses are updated.

2.2.2 Non-functional Requirements:

- User preference updates must be processed within 2 seconds.
- The service must ensure that only the owner of the profile can modify their data.
- The service must be scalable to support multiple users updating preferences simultaneously.

2.3 Availability Management Service

2.3.1 Functional Requirements:

- The system must allow users to define their study availability during the week.
- The system must allow users to update or view their current availability schedule
- The system must allow users to delete existing availability slots.
- The service must publish an event when availability is created or updated so that matching can be recalculated.
- The system must ensure users cannot create overlapping availability entries.

2.3.2 Non-functional Requirements:

- The system must ensure data accuracy when storing time slots.
- The service must support simultaneous updates from multiple users without data corruption.

2.4 Matching Service

2.4.1 Functional Requirements:

- The system must analyze user information including:
  1. courses
  2. topics
  3. availability
  4. study preferences
- The system must generate recommended study buddies based on compatibility.
- The system must assign a compatibility score to potential matches.
- The system must allow users to view a list of recommended study partners.
- The service must publish an event when a match is identified.

2.4.2 Non-functional Requirements:

- The matching process must return recommendations within 3 seconds.
- The system must support multiple matching requests simultaneously.
- The service must ensure fair and consistent matching results based on available data.

2.5 Study Session Scheduling Service

2.5.1 Functional Requirements:

- The system must allow users to create a study session.
- The system must allow users to join existing sessions.
- The system must allow users to cancel or leave a session.
- The system must store session details including:
  1. topic
  2. date and time
  3. duration
  4. session type (online or in-person meeting)
  5. contact info of the session creator and receiver in case there is no messaging service
- The system must notify other services when sessions are created or updated.

2.6 Notification Service

2.6.1 Functional Requirements:

- The system must notify users when important system events happen.
- Notifications must be created when:
  1. a match is found
  2. a study session is created
  3. a session invitation is received
- The system must allow users to view their notifications.
- The system must allow users to mark notifications as read.

2.7 Messaging / Chat Service (Bonus Feature)

2.7.1 Functional Requirements:

- The system must allow matched users to send messages to each other.
- The system must store a history of conversations.
- The system must allow users to view previous messages in a conversation.

2.8 Integration & Deployment

- The system shall be deployable on cloud platforms like Render,Railway,or Fly.io.
- The system shall integrate Kafka for real-time updates and async communication between services.
- Docker will be used for deploying the system.
  More details will be provided on what will exactly be deployed on Docker.

3 Milestone 1: UI/UX Design in Figma (30%)

3.1 Deadline

This milestone should be submitted before 17/3/2026

3.2 Objective

Design a user-friendly interface in Figma for the Study Buddy Matcher platform. The design should clearly demonstrate the interaction between different pages and UI components. The prototype must represent the complete user journey from registration to finding study buddies and organizing study sessions, ensuring all interactions between the pages/UI components are shown. The design must incorporate all UI principles covered in the labs.

3.3 Requirements

3.3.1 Figma Design for Every Page:

- **Landing / Welcome Page**
  Introducing the platform, explaining its purpose, and allowing users to sign up or log in.
- **User Authentication**
  Registration and login pages where students create accounts and access the system.
- **User Profile Setup**
  A page where users enter their academic information such as university, academic year, courses, and study topics.
- **Study Preferences Setup**
  Users should be able to specify how they prefer to study, including:
  - preferred study pace
  - preferred study mode (online or in-person)
  - preferred group size
  - preferred study style (writing, listening, quiet study, discussion, etc.)
- **Availability Management Page**
  Users define the time slots during which they are available for study sessions.
- **Dashboard / Home Page**
  A central page that summarizes the user's activity and provides access to main features such as:
  - recommended study buddies
  - upcoming study sessions
  - notifications
  - quick navigation to profile, matching, and sessions
- **Matching / Study Buddy Recommendation Page**
  A page displaying suggested study partners based on shared courses, preferences, and availability.
- **Match Details Page**
  A page showing detailed information about a potential study buddy, including shared courses, study preferences, and overlapping availability.
- **Buddy Requests / Connections Page**
  A page where users can view incoming and outgoing study buddy requests and manage accepted study partners.
- **Study Sessions Page**
  A page displaying upcoming and past study sessions that the user has created or joined.
- **Create Study Session Page**
  A form allowing users to create new study sessions by selecting:

  - topic
  - date and time
  - duration
  - session type (online or in-person)
  - participants

- **Notifications Page**
  A page or dropdown panel showing notifications such as:
  - new match found
  - buddy request received
  - session invitation
  - session reminder

- **Messaging / Chat Page (Optional Feature)**
  A page allowing matched users to communicate and coordinate study sessions.

- **User Profile & Activity Page**
  A page where users can view and edit their profile information, study preferences, and view their past sessions or connections.

- **Document stating what concepts you applied in each frame/page**
  Students must provide a short document explaining the UI/UX concepts used in each screen, such as layout decisions, usability considerations, and interaction design.

3.4 Submission

The submission will be through a google form where you will send a drive link that includes the figma design/prototype.

4 Milestone 2 : Backend Development & Service Integration (40%)

4.1 Deadline:

This milestone should be submitted before 10/4/2026

4.2 Objective:

This milestone focuses on building the backend with **microservice-based** architecture using Node.js where each service has its own DB taht will be set on **NeonDB and Prisma**. It also includes setting up **Kafka** for event-driven communication between services. The backend should expose a **GraphQL API** to allow the frontend to interact with the system efficiently.

4.3 Requirements:

4.3.1 Microservices-Based Backend :

The project should be divided into the following services:

4.3.2 Microservices-Based Backend

The backend system should follow a **microservices architecture**, where each service is responsible for a specific functionality of the platform. Services should communicate with each other through **Kafka events** and expose APIs that can be accessed through the GraphQL Gateway.

The project should be divided into the following services:

- **User Service**
  This service manages user authentication and basic account information. It is responsible for user registration, login, and account management. The service should store essential user data such as name, email, university, and academic year. It should also generate authentication tokens (e.g., JWT) to allow secure access to the system. In addition to authentication data, this service should allow users to provide **contact information** such as an email address or phone number. This allows matched study buddies to communicate with each other outside the platform in case the optional Messaging Service is not implemented.

- **Profile & Preferences Service**
  This service manages the academic and study-related information of users. It allows users to define the courses they are currently studying, the topics they need help with, and their preferred study behavior. Preferences may include study pace, study mode (online or in-person), preferred group size, and study style (e.g., writing notes, discussion, quiet study). These preferences are important inputs for the matching process.

- **Availability Service**
  This service allows users to define and manage their weekly study availability. Users should be able to add, update, and remove time slots indicating when they are free to study. The availability data is used by the matching service to determine whether two users have overlapping free time and can realistically schedule study sessions together.

- **Matching Service**
  This service is responsible for generating study buddy recommendations. It consumes events from other services (such as profile updates or availability updates) and stores the necessary information for matching. The service compares users based on shared courses, shared topics, overlapping availability, study preferences, and study styles.

  The service should calculate a **compatibility score** for each potential match using a rule-based scoring system. The final score can be normalized to a value out of **100**, and the system should return the highest ranked candidates.

  Optionally, the service may also return short explanations describing why two users were matched (for example: shared course, similar study style, or overlapping availability).

- **Study Session Service**
  This service manages the creation and organization of study sessions. Users should be able to create study sessions by specifying details such as topic, date, time, duration, and meeting type (online or in-person). The service should also allow users to join sessions created by other users and manage participant lists. It must ensure that session information is stored correctly and can be retrieved through the GraphQL API.

- **Notification Service**
  This service is responsible for informing users about important events in the system. It listens to **Kafka events** generated by other services and creates notifications for users. Examples include notifications for new study buddy matches, received buddy requests, session invitations, and upcoming session reminders.

- **Messaging Service (Bonus)**
  This service enables communication between users who are matched or participating in the same study session. It allows users to send messages, view previous conversations, and coordinate study sessions.

  The service should support the following features:

  - sending messages between users
  - storing conversation history
  - retrieving previous messages

  Messages should be stored in the database with basic information such as sender ID, receiver or conversation ID, message content, and timestamp.
  If this service is not implemented, you have to allow users in the same session to communicate with each other using contact info.

4.3.3 Database Setup (NeonDB + Prisma)

You will setup your database (neonDB) along with Prisma.

- PostgreSQL database hosted on **NeonDB** and managed using **Prisma ORM**.
- Schema definitions for the main entities in the system, including:
  - users
  - user profiles and study preferences
  - availability time slots
  - study sessions
  - buddy requests or connections
  - matching
  - notifications
  - messaging (bonus)
- The schema design should support relationships between users, their study preferences, availability schedules, and study sessions in order to enable the matching and scheduling functionalities of the system.

4.3.4 GraphQL API

Create a GraphQL gateway that exposes queries and mutations to interact with the system.

Examples include:

- Queries:
  - retrieving user profiles
  - retrieving recommended study buddies
  - retrieving study sessions
  - retrieving notifications
- Mutations:
  - updating user preferences
  - updating availability
  - sending buddy requests
  - creating study sessions
  - joining study sessions

Note: GraphQL will also be implemented for the frontend (apollo client).

4.3.5 Message Broker for Service Communication (Kafka)

Kafka will be used as the **message broker** to enable event-driven communication between the microservices. Instead of services calling each other directly, important system actions should produce events that other services can consume and react to asynchronously.

Each service may act as a **Kafka producer**, a **Kafka consumer**, or both. When a significant action occurs in the system, the responsible service should publish an event to Kafka. Other services that subscribe to that event can then perform additional actions.

Typical events in the system may include:

- `UserPreferencesUpdated`
- `AvailabilityUpdated`
- `BuddyRequestCreated`
- `StudySessionCreated`
- `StudySessionJoined`

For example:

- When a user updates their availability, the Availability Service publishes an event.
- The Matching Service consumes this event and recalculates possible matches.
- When a new match is generated, another event can be produced.
- The Notification Service consumes the event and generates a user notification.

User actions such as sending buddy requests, creating study sessions, or joining sessions should first be triggered through the **GraphQL API**. After the action is successfully processed and stored in the database, the corresponding service should publish a Kafka event so that other services can react accordingly.

Each event message should include structured information such as:

- event name
- timestamp
- producer service
- correlation ID
- payload data

This structure ensures that events can be traced, monitored, and consumed reliably by other services in the system.

- Users receive booking confirmations via email & SMS.
- Ride status updates sent in real time.
- Admin notifications for disputes or payment issues.

4.4 Submission

The submission will be through a google form too where you will submit the backend of your system through a zip folder.

4.5 Evaluation

There will be an evalution for this milestone where you will receive feedbacks about the project. You will reserve slots based on a first-come first-serve basis.

5 Milestone 3: Frontend Implementation & Deployment (30%)

5.1 Deadline:

This milestone should be submitted before 12/5/2026

5.2 Objective

This milestone focuses on implementing the UI design (from M1) in React, integrating it with the backend (from M2) using GraphQL client (Apollo) , and deploying the complete system on Render/Railway and Docker.

5.3 Requirements

5.3.1 Frontend Implementation in React

- Convert Figma designs (from M1) into code.
- Implement the pages and their functionalities.
- Optimize UI/UX for performance and responsiveness.

5.3.2 Integration with Backend (Node.js)

- Connect React frontend with GraphQL API.
- Ensure proper state management and API communication.

5.3.3 Deployment

- Deploy Project(Render or Railway and Docker)
- Ensure the database (NeonDB) is accessible from deployed services.

Note: you need to deploy your project on cloud platforms that support Docker.

5.3.4 Evaluation

There will be a final evaluation for the whole project where you will show the full functionality of your system. Feedback from M2 should be taken into account.

6 Important Notes

- AI-generated code is allowed, **BUT** you should understand the code. If not, a ZERO will be given in the whole project.
- The suggested hosting and deployment sites are up to you. You can use whatever you prefer.
- **This document might be updated; however, you will be informed whenever this happens.**
- If you reached this far, congratulations! You are done with the first step towards completing this project :D! Best of luck! I'm excited to see your creativity in action! You got this!